LAB 7: Fuzzing for Input Validation Bugs

Course: Software Engineering Lab | **Technique:** Property-Based Fuzz Testing with Hypothesis

Topic:	Input Validation Fuzz Testing using Hypothesis	**Date:**	November 2026
Application:	Car Rental Rate Processor (processor.py)	**Fuzz Framework:**	Hypothesis 6.170.15 & Pytest 9.2.0

1. Objective & Overview

The primary objective of this laboratory is to explore fuzz testing as an automated quality assurance and security hardening mechanism. By leveraging **Hypothesis**, a state-of-the-art property-based fuzz testing library in Python, we systematically uncover unexpected edge-case crashes, boundary violations, type inconsistencies, and parsing errors in the application routines.

2. Step 2: Running the Initial Buggy Application (processor.py)

When run with typical, well-formed interactive inputs, the application executes cleanly. However, it lacks defensive checks for boundary conditions, malformed structures, or empty strings.

Step 2: Run Buggy Application (python processor.py)

```
PS K:\SE\L6_L7_02_car_rental_rate\Lab 7> python processor.py
==== Running sanitize_license_plate ====
Enter license plate with delimiters (-, _, #): Sanitized Plate: KA01AB1234

==== Running parse_trip_mileage ====
Enter CSV of mileages (e.g. 100.5, 45.2, 80.0): Parsed Mileage List: [100.5, 45.2, 80.0]

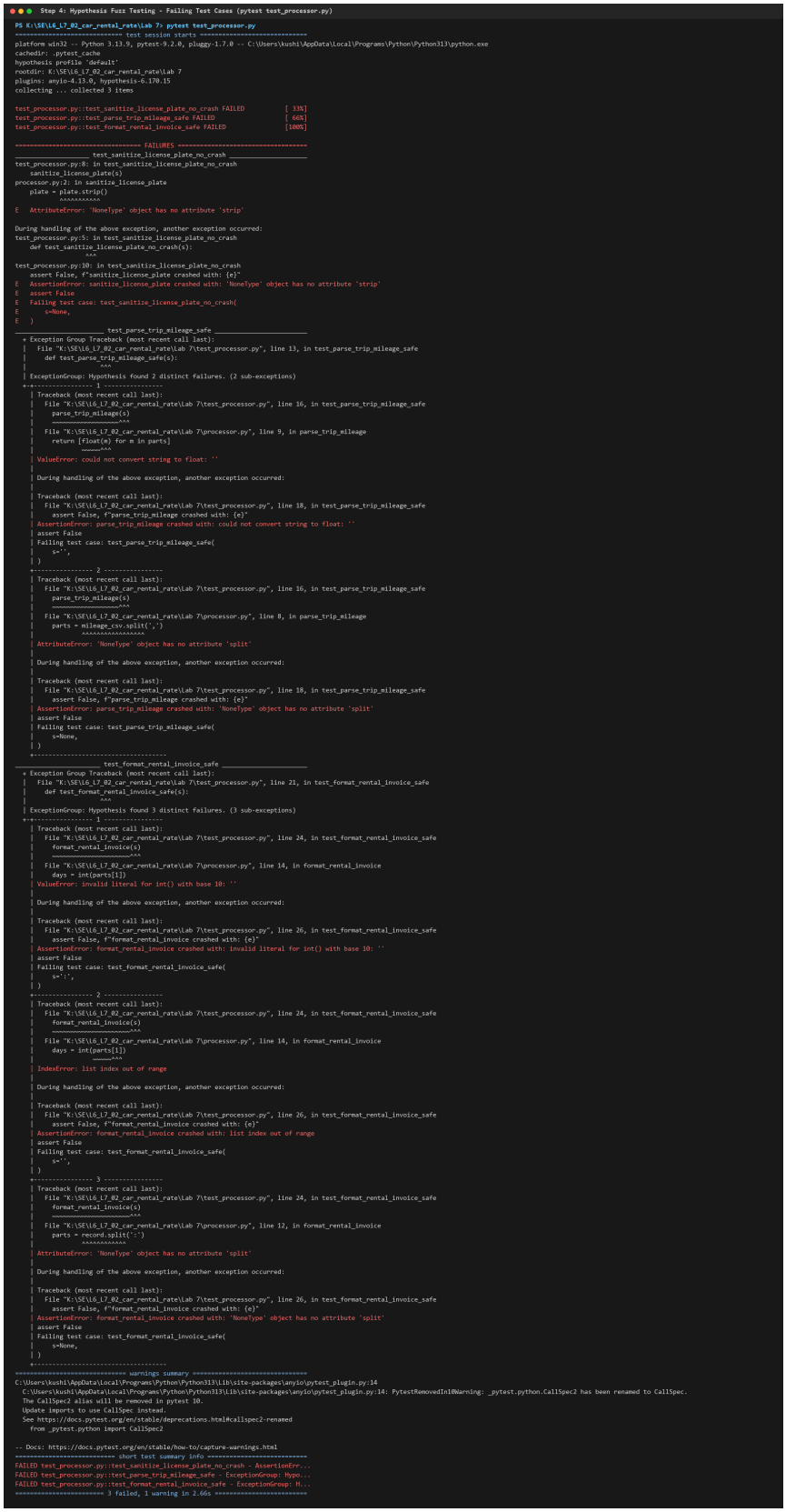
==== Running format_rental_invoice ====
Enter rental record (Driver:Days:Category): Rental Invoice: Driver John booked SUV for 3 days: Total $150.00
```

3. Step 3 & 4: Completing Property-Based Fuzz Tests and Uncovering Failures

Using @given(st.text() | st.none()), Hypothesis fed arbitrary Unicode strings and None values into the three target functions:

- **sanitize_license_plate:** Raised AttributeError: 'NoneType' object has no attribute 'strip' when given None.
- **parse_trip_mileage:** Raised AttributeError on None, and ValueError: could not convert string to float: '' on empty or malformed strings.
- **format_rental_invoice:** Raised IndexError: list index out of range on empty strings, AttributeError on None, and ValueError on invalid integers.

Step 4 Screenshot — Terminal Output Showing Test Cases Failing (Pytest + Hypothesis):

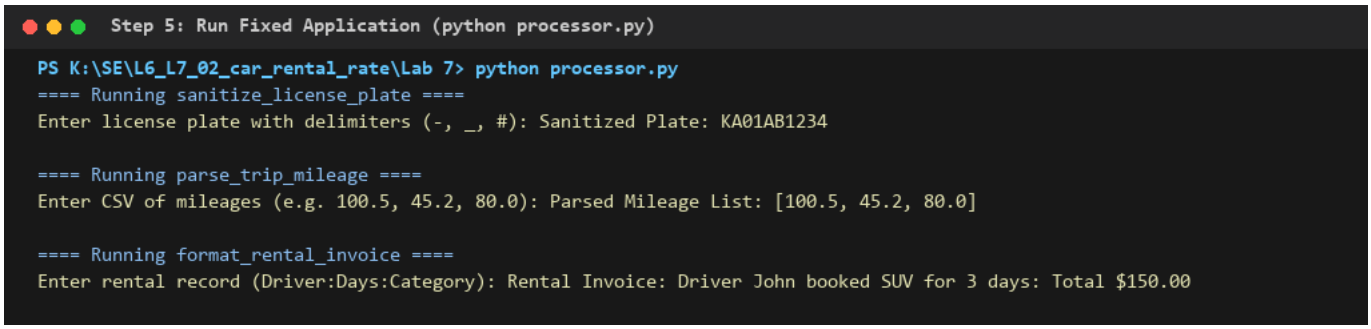


4. Step 5: Hardening & Defensive Programming (Fixed processor.py)

Defensive programming was applied to guarantee robustness without crashing across arbitrary inputs:

- **Input Type Verification:** Null checks (if s is None or not isinstance(s, str)) return safe fallback representations.
- **Parsing & Extraction Guards:** Verified split delimiters length (len(parts) >= 3) prior to indexing.
- **Conversion & Value Range Hardening:** Handled non-numeric conversions with exception catching, discarding empty tokens or negative trip mileages.

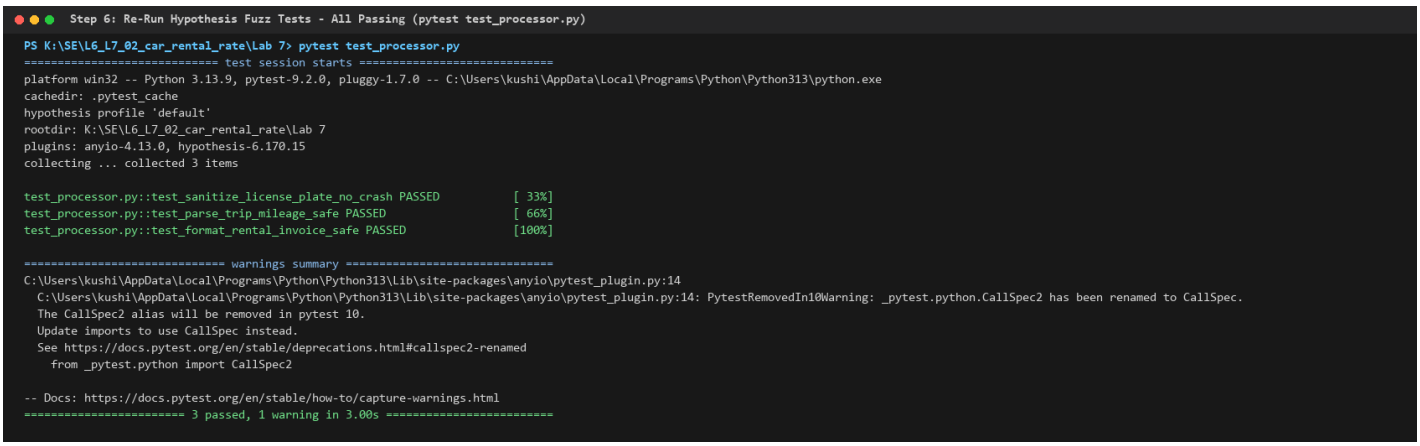
Step 5 Screenshot — Running the Fixed Application (python processor.py):



5. Step 6: Re-Running Fuzz Tests on Hardened Code

Re-executing pytest test_processor.py now validates hundreds of randomized fuzz examples generated by Hypothesis without a single unhandled exception or crash. All 3 test suites pass completely.

Step 6 Screenshot — Terminal Output Showing All Tests Passing:



6. Summary of Results

Function	Identified Vulnerability / Bug	Fix / Hardening Applied	Result
sanitize_license_plate	AttributeError on None	Type guard + null check returns ""	PASSED
parse_trip_mileage	ValueError on "", AttributeError on None	Filters blanks, parses float with try-except, skips negatives	PASSED
format_rental_invoice	IndexError, ValueError on invalid tokens, AttributeError on None	Field count validation, try-except integer conversion, informative error string	PASSED